BRI-410: A 512-bit Tweakable Block Cipher with Key- and Tweak-Dependent Permutations

Rinald Cobaj, Independent researcher, vlorarino@gmail.com

Paper version 2 (2026-10-27), describing cipher version BRI-410 v1.1. BRI-410 is a study cipher: it has not been analysed by independent cryptanalysts and must not be used to protect real data. Code, test vectors and all analysis scripts: https://github.com/rinaldcobaj-cell/BRI-410

Abstract

BRI-410 is an experimental tweakable block cipher with a 512-bit block, a 410-bit key, a 128-bit tweak and 16 rounds. It comes with BRI-410 AE, an instantiation of the ΘCB3 authenticated-encryption mode with a 416-bit tag. BRI-410 is a key-alternating substitution–permutation network on an 8×8 array of bytes. It uses an 8-bit S-box affine-equivalent to the AES S-box and an 8×8 Cauchy MDS matrix. Its distinctive feature is the permutation layer: in every round, the key and the tweak select a cell permutation from a family of 32 "knight-move" permutations, all of which are diffusion-optimal in both directions. We prove that the wide-trail bound of 81 active S-boxes over four rounds holds for every sequence of permutations from this family. For individual differential characteristics and linear trails, this gives at least 324 active S-boxes over 16 rounds. A CP-SAT model confirms the values for one to five rounds. This bound is not a proof of the overall security of BRI-410. All round keys and permutations are derived from the key and the tweak. Version 1.1 adds a Davies–Meyer feed-forward after tweak injection, intended to make the per-block derivation one-way, because in version 1.0 three consecutive round keys of a single block revealed the master key in 0.1 seconds. That one-wayness is a conjecture, not a proven property. We document this weakness and the other negative results of the design process, among them a predecessor with 4-bit cells that we broke over 5 rounds. We report experiments in five categories of cryptanalysis, keep proofs, computations, experiments and conjectures apart, and list open problems for independent analysis. We do not claim any practical advantage over existing designs.

1. Introduction

The rules of chess are public, yet a serious game almost never repeats. BRI-410 starts from this analogy: the algorithm is public, and the key, a "game", is secret. In addition, every encrypted block plays its own game, derived from the key and a per-block tweak. The 8×8 board fixes the state. The knight's move gives the permutation layer. The key length comes from Allis's estimate of about 10^123 games of chess [All94], since log2(10^123) ≈ 408.6. **This choice is symbolic.** We do not claim that security above 256 bits has practical meaning, and our security target (Section 9) is lower than 2^410.

The aim of this paper is not to propose a replacement for existing ciphers. It describes BRI-410 completely enough that independent cryptanalysts can analyse it, challenge it and, if possible, break it. It also records every weakness we found during the design.

Contributions.

1. A complete specification of BRI-410 v1.1, a 512-bit tweakable block cipher with a 410-bit key and 128-bit tweak, and of BRI-410 AE (Sections 3 and 4).
2. A proof that the two-round (9) and four-round (81) wide-trail bounds on individual differential characteristics and linear trails hold for every key- and tweak-dependent choice of the round permutations, both for the cipher and for the key schedule (Section 5).

3. A record of the design process with its negative results. These include a useless parameter (α), a key-dependent "game" that hides only one bit per round at row level, a 4-bit-cell predecessor broken over 5 rounds, and an invertible key schedule in version 1.0, together with the change that addresses it (Sections 7 and 10).
4. Experiments in five categories: differential, linear, algebraic, key schedule, and chosen plaintext/tweak (Section 6). Open problems and concrete targets for analysis are in Section 12.

Kinds of evidence. Every statement is labelled:

- **[Proof]** a mathematical proof given in this paper
- **[Computation]** an exact computer computation (exhaustive check or solver)
- **[Experiment]** a statistical or sample-based measurement
- **[Estimate]** a complexity estimate that was not implemented
- **[Conjecture]** a hypothesis we believe but cannot support with proof
- **[Open]** a question we did not study

2. Related work and positioning

Tweakable block ciphers. Liskov, Rivest and Wagner introduced tweakable block ciphers [LRW02]. The TWEAKEY framework [JNP14] processes the key and the tweak with a light, linear schedule. It underlies SKINNY [BJK+16] and Deoxys-BC [JNPS21], which have 64- and 128-bit blocks and extensive public analysis. Threefish [FLS+10], the block cipher of Skein, is the closest design in size: it has 256-, 512- and 1024-bit blocks and a 128-bit tweak, but it is an ARX design (addition, rotation, XOR) with 72 rounds for the 512-bit block.

Wide-block SPNs. Rijndael supports 256-bit blocks [DR02]. Kalyna [OGK+15] is an SPN standard with 512-bit blocks and keys, 8-bit S-boxes and 8×8 MDS matrices over GF(2^8), but without a tweak.

Key-dependent components and rekeying. Blowfish [Sch93] and Twofish [SKW+98] derive S-boxes from the key. Deriving a fresh key for every message or block is a known technique, analysed for example in [AB00]. The Davies–Meyer construction from hash functions adds a feed-forward so that the output is believed hard to invert when the underlying permutation is strong [PGV93].

Cipher	Block	Key	Tweak	Rounds	Structure	Tweak processing
AES-256	128	256	—	14	SPN, 8-bit S-box, MDS	—
Kalyna-512/512	512	512	—	18	SPN, 8-bit S-boxes, MDS	—
Threefish-512	512	512	128	72	ARX	added to subkeys
SKINNY, Deoxys-BC	64 / 128	(tweakey)	(tweakey)	14–56	SPN	linear TWEAKEY schedule
BRI-410 v1.1	**512**	**410**	**128**	**16**	**SPN, 8-bit S-box, MDS**	**full re-derivation of all round keys and permutations, intended to be one-way (Davies–Meyer, not proven)**

Positioning. BRI-410 brings no gain in efficiency. Re-deriving the whole schedule for every block is slower than TWEAKEY (Section 11). What we study is a different design point: a permutation layer that depends on the key and the tweak, chosen in every round from a family that is diffusion-optimal for every choice. As a result, the classical wide-trail bounds hold uniformly over all keys and tweaks. To our knowledge this exact combination has not been described before. Key-dependent components and tweak-dependent round keys as such are well known.

3. Specification of BRI-410 v1.1

3.1 Notation and parameters

Parameter	Value
Block	512 bits = 64 bytes. Byte i is cell (r, k) = (⌊i/8⌋, i mod 8) of an 8×8 board; row r = bytes 8r..8r+7
Key	410 bits, given as 52 bytes little-endian whose 6 top bits are 0
Tweak	128 bits (16 bytes)
Rounds	R = 16
Field	GF(2^8) with polynomial x^8 + x^4 + x^3 + x + 1

3.2 Components

S-box S. `S(x) = A·x^-1 ⊕ c` (with 0^-1 = 0). Here `A` is an invertible 8×8 binary matrix: row i is a byte, and bit i of `A·y` is the parity of `A[i] AND y`. `c` is a byte. `A ‖ c` are the 9 bytes of `SHAKE-256("BRI-410 S-box8 v1" ‖ n)`, where n is the first 4-byte big-endian counter that gives an invertible A and an S-box without points S(x) = x or S(x) = x ⊕ FF. The result is n = 9. The table is in Appendix A.

Permutation family π. For (dr, dk) ∈ D = ((1,2), (1,−2), (−1,2), (−1,−2)), in this order, and β ∈ {0,…,7}:

```
π_(d,β):  cell (r, k)  →  cell (r + k·dr,  k + k·dk + β)   (mod 8)
```

Cell (r, k) makes k knight jumps (dr, dk) on a toroidal board, then the board shifts β columns. This gives 32 permutations, all distinct.

Linear layer θ. Each row is multiplied by the 8×8 matrix `M[i][j] = 1 / (x_i ⊕ y_j)`, where x = (0,…,7) and y = (145, 234, 150, 206, 98, 226, 235, 94), the first 8 distinct bytes outside {0,…,7} of `SHAKE-256("BRI-410 MDS8 v1")`. M and M^-1 are in Appendix A.

3.3 Encryption and decryption

```
X ← P ⊕ RK_0                            (whitening with RK_0, Section 3.4)
for i = 1 .. 16:
    X ← θ( π_i( S(X) ) ) ⊕ RK_i         (S on all 64 bytes; π_i moves S(X)[c] to cell π_i(c))
C ← X
```

Decryption applies `⊕RK_i, θ^-1, π_i^-1, S^-1` for i = 16, …, 1, and finally `⊕RK_0`. All rounds are identical. The last round also applies θ.

3.4 Key and tweak schedule ("the game")

The schedule state G has 80 bytes (10 rows of 8). One step is

```
F_c(G):  S on all 80 bytes → θ on each of the 10 rows → byte j moves to position 9·j mod 80 → byte 79 ⊕= c
```

Version 1.1 computes:

```
L   ← key (bits 0..409) ‖ first 230 bits of SHAKE-256("BRI-410/8 mbushja") as little-endian (bits 410..639)
K*  ← F_0xF3(F_0xF2(F_0xF1(F_0xF0(L))))           key only; the same for every tweak
X   ← K* with bytes 0..15 XORed with the tweak
G_0 ← F_0xFB(…F_0xF4(X)…) ⊕ X                     8 steps and a Davies–Meyer feed-forward
RK_0 ← bytes 0..63 of G_0                          the whitening key of Section 3.3
for i = 1 .. 16:
    G_i ← F_i(G_(i−1))
    RK_i ← bytes 0..63 of G_i
    π_i  ← π_(D[b & 3], (b >> 2) & 7), where b = byte 64 of G_i
```

So there are 17 round keys, RK_0 to RK_16, and 16 permutations, π_1 to π_16. The schedule cost is (each step evaluates 80 S-boxes):

Process	Steps	S-boxes
Key preparation K*, once per key and shared by all tweaks	4	320
Tweak mixing, per tweak	8	640
Round-key generation, per tweak	16	1 280
Schedule for one tweak, without the shared preparation	24	1 920
Schedule for the first tweak under a key, including the preparation	28	2 240

With the 1 024 S-boxes of the 16 rounds, the first block under a key costs 3 264 S-boxes, and every further block, which reuses K*, costs 2 944. The feed-forward is a design measure meant to make the map from G_0 back to X hard to invert. **We do not prove that it is one-way** (Conjecture 2 in Section 9).

Version 1.0 (Sections 6 to 8, history) used 4 steps after the tweak and no feed-forward: `G_0 = F_0xF7(…F_0xF4(X)…)`, which is 4 + 16 = 20 steps per tweak after the shared preparation, or 24 including it. The round function is the same in both versions.

4. BRI-410 AE

BRI-410 AE instantiates ΘCB3 [KR11] with BRI-410/8 v1.1 as the tweakable block cipher E. Each call uses the tweak `N ‖ ((index << 4) | type)`: a 12-byte nonce N, then 4 bytes little-endian, with index < 2^28. The types are 0 message block, 1 mask of the final partial block, 2 tag without a partial block, 3 tag with a partial block, 4 associated-data block, and 5 final partial associated-data block. Associated data uses N = 0^12. With M = M_1 … M_m M_* and A = A_1 … A_a A_* (|M_*|, |A_*| < 64 bytes), and pad(x) = x ‖ 80 ‖ 00…:

```
C_i   = E^(N,i,0)(M_i)                                                    i = 1..m
if M_* is not empty:  C_* = M_* ⊕ E^(N,m,1)(0^512) truncated;   F = E^(N,m,3)(⊕M_i ⊕ pad(M_*))
otherwise:            F = E^(N,m,2)(⊕M_i)
Auth  = ⊕ E^(0,i,4)(A_i)   [⊕ E^(0,a,5)(pad(A_*))]
Tag   = first 52 bytes of (F ⊕ Auth)                                       416 bits
Packet = N ‖ C ‖ Tag
```

Decryption returns nothing unless the tag verifies, and the comparison is constant-time. The correspondence with ΘCB3 was checked by the author against the description of the mode, not by an independent party. **[Open]**

5. Structural results

Lemma 1 (permutation family). [Proof] Each π_(d,β) is a bijection, and it is diffusion-optimal in both directions: the 8 cells of any row go to 8 distinct rows, and the 8 cells that land in any row come from 8 distinct rows.

Proof. The column map k → k(1 + dk) + β is a bijection of Z_8, since 1 + dk ∈ {3, −1} is odd. Given the column, the row map r → r + k·dr is a bijection, so π is a bijection. For a fixed source row r, the destination rows r + k·dr (k = 0..7) are distinct because dr = ±1 is invertible mod 8. For a destination row q, every column k occurs exactly once among its cells, because the column map is a bijection. The source row of the cell with column k is q − k·dr, and these are distinct. ∎

Lemma 2 (the row shift α adds no structure). [Proof] Let T_α be the row rotation (r, k) → (r + α, k). Then T_α commutes with S, which acts cell by cell, with every π_(d,β), since both sides give (r + α + k·dr, k(1+dk) + β), and with θ, which applies the same matrix to every row. Hence a round-dependent T_α inside the round can be moved to the end of the cipher, replacing RK_i with T(RK_i). So it only adds a final rotation of rows that depends on the key. We therefore removed α (in an earlier version, measurements found 1 792 pairs of "game" variants that were equivalent up to α). ∎

Lemma 3 (MDS). [Proof + Computation] Every square submatrix of a Cauchy matrix is a Cauchy matrix, and its determinant is non-zero when the x_i are distinct, the y_j are distinct and x_i ≠ y_j [MS77]. Hence M is MDS and its differential branch number is 9. Since the dual of an MDS code is MDS, the linear branch number is also 9. We also checked all 12 869 square submatrices of M and of M^-1: none is singular. M has no column symmetry other than the identity.

Theorem 1 (bounds for every permutation sequence). [Proof] Consider any differential (or linear) trail through r rounds of BRI-410 with a non-zero input, any key and any tweak. Then: (a) any two consecutive rounds contain at least 9 active S-boxes, and (b) any four consecutive rounds contain at least 81 active S-boxes, for every sequence of permutations π_i from the family.

Proof. Let a_t be the number of active bytes at the input of the S-layer of round t, and let ρ(·) be the number of active rows. (a) θ_t acts row by row with branch number 9. For each active row at its input, active input bytes plus active output bytes ≥ 9. The input bytes of θ_t are the S-outputs of round t, moved by π_t. Its output bytes are the S-inputs of round t+1, since adding the key does not change activity. Summing over rows gives a_t + a_(t+1) ≥ 9 · ρ(input of θ_t) ≥ 9. (b) Take rounds 1 to 4. By (a), a_1 + a_2 ≥ 9·ρ_1 and a_3 + a_4 ≥ 9·ρ_3, where ρ_t is the number of active rows at the input of θ_t. Let q be any active row at the input of θ_2. It has w_in active bytes, and its image row at the output of θ_2 has w_out bytes, with w_in + w_out ≥ 9. The w_in bytes come, through π_2, from w_in distinct rows of the state at the input of round 2 (Lemma 1, inverse direction). Each of these rows is an output row of θ_1, so it is also active at the input of θ_1, hence ρ_1 ≥ w_in. The w_out bytes of row q are S-inputs of round 3, and π_3 sends them to w_out distinct rows (Lemma 1, forward direction), hence ρ_3 ≥ w_out. Therefore a_1 + … + a_4 ≥ 9(ρ_1 + ρ_3) ≥ 9(w_in + w_out) ≥ 81. The argument for linear trails is the same, with masks propagating through the transpose of θ, which also has branch number 9 (Lemma 3). ∎

Corollary. 4k rounds contain ≥ 81k active S-boxes. 16 rounds contain ≥ 324.

Theorem 2 (key and tweak schedule). [Proof + Computation] The permutation j → 9j mod 80 of the schedule sends the 8 bytes of each of the 10 rows to 8 distinct rows, and receives each row from 8 distinct rows [Computation]. The proof of Theorem 1 therefore applies to the steps F_c. Any non-zero difference in the schedule state, in particular a difference in the tweak, activates at least 81 S-boxes in 4 steps and at least 162 in the 8 steps of v1.1 that precede RK_0.

What these bounds guarantee, and what they do not.

- They **guarantee** that every single differential characteristic, or linear trail, activates at least the stated number of S-boxes, for every key, every tweak and every choice of permutations. With the maximal S-box probabilities of 2^-6, this means that any single 4-round trail has probability ≤ 2^-486 and any 16-round trail ≤ 2^-1944, **under the usual assumption of independent round keys**.
- They **do not guarantee** anything about (i) differentials, where many trails add up, (ii) linear hulls, (iii) the actual dependencies between round keys produced by the schedule, or (iv) attacks of other kinds: integral, algebraic, boomerang, impossible-differential, or related-key beyond the schedule bound.
- In particular, Theorems 1 and 2 are **not a proof of the security of BRI-410 as a whole**.

Computation. [Computation] A CP-SAT model at the level of active bytes, in which the adversary may choose the row structure of π_i in every round (Appendix C), returned the minimum number of active S-boxes: 1, 9, 17, 81 and 82 for 1 to 5 rounds (proven optimal), and 1, 9, 17, 81, 107 and 114 for 1 to 6 schedule steps. For 6, 7 and 8 rounds it found trails with exactly 90, 98 and 162, which equal the bounds that follow from Theorem 1. These values were computed once (OR-Tools 9.15, 2026-10-22). Later runs on the author's machine were blocked by an operating-system application-control policy, so an independent re-run is welcome. The proofs above do not depend on this computation.

6. Experiments in five categories

All experiments use fixed random seeds and are in the repository. Experiments marked "v1.0" were run before the schedule change. Experiments that only involve the round function under a fixed tweak (D, L, A) do not depend on how the round keys are derived. Experiments K and T were repeated for v1.1 (Section 7).

6.1 Differential (D)

Measurement	Result	
S-box difference table (255 × 256 entries)	32 895 × 0, 32 130 × 2, 255 × 4; maximal probability 2^-6	[Computation]
One round: most frequent output difference (3 × 2^14 pairs)	frequency 0.0161 / 0.0157 / 0.0142, against 4/256 = 0.0156 predicted; 127 distinct outputs, as predicted	[Experiment]
One active input byte: number of output bytes with zero difference (300 pairs; random 0.25)	1 round: 56 · 2 rounds: 0 in every pair · 3 rounds: 0.29	[Experiment]

After 2 rounds every byte is always active, which distinguishes 2 rounds from a random permutation, as for AES. We saw no deviation after 3 rounds.

6.2 Linear (L)

Measurement	Result	
S-box: maximal correlation	32/256 = 2^-3	[Computation]
One round, exact: only byte 0 varies (256 inputs), output byte 0	maximal correlation 0.125, equal to the S-box. A byte of another row does not change at all	[Computation]

Negative methodological result. We first ran a statistical test that correlates one input byte with one output byte over 2^18 random plaintexts. It is **uninformative by construction**: with all other input bytes random, an output byte is the XOR of 8 S-box outputs from 8 different input bytes, so the correlation is zero even after one round. We do not use it as evidence. Linear trails over two or more rounds have correlation ≤ 2^-27 and cannot be measured experimentally. **The linear security argument rests on Theorem 1.**

6.3 Algebraic (A)

Measurement	Result	
Linearly independent quadratic equations relating input and output of S	39, equal to the AES S-box with the same code	[Computation]
Linear structures of S	0	[Computation]
Size of a polynomial system that expresses one block in terms of the master key, v1.1	3 264 S-boxes (1 024 in the cipher and 2 240 in the schedule, the 4 key-only steps included; Section 3.4): ≈ 127 000 quadratic equations (39 per S-box) in ≈ 26 500 variables (8 per S-box output plus 410 key bits). Further blocks under the same key add 2 944 S-boxes each. This is a count, not an attack	[Computation]

Cube tests [Experiment] (XOR-sum of the outputs over all values of d chosen plaintext bits, fixed tweak; the number of output bits whose sum is zero, where random gives about 256 of 512):

Cube	2 rounds	3 rounds	4 rounds
8 bits in one byte	512, 512	512, 512	241, 264
8 bits in 8 different bytes	512, 512	236, 263	247, 255
12 bits in 12 bytes	512, 512	259, 234	258, 248
16 bits in 16 bytes	512, 512	512, 257	258, 232

Explanation of the 3-round zero sums [Proof]. In round 1 each S-box sees at most one variable bit, so its output is affine in that bit. If π_1 puts at most k cube bits in each row, the degree after round 2 is at most k and after round 3 at most 7k. With 8k cube bits in total and 7k < 8k, the sum is zero. We observed no zero sums over 4 rounds.

6.4 Key schedule (K)

Measurement	Result	
One key bit flipped: bits changed in RK_0, RK_1, RK_8, RK_16 (v1.1, 820 trials)	50.1%, 50.1%, 50.1%, 50.0%	[Experiment]
Permutation changes when one key bit flips (v1.0)	97.0%, against 1 − 1/32 = 96.9% expected	[Experiment]
Structured keys (zero, all ones, single bit, 0x55…, counting; v1.0)	round keys 50–51% ones; 3 000 output blocks each: monobit z ≤ 1.98, byte χ^2 218–275 (99% threshold 310.5)	[Experiment]
Recovery of the master key from round keys	v1.0: broken. v1.1: blocked for the direct method (Section 7)	[Experiment]

6.5 Chosen plaintext and chosen tweak (T)

Measurement	Result (random: 0.25 bytes)	
Tweak integral, v1.1 (one tweak byte takes all 256 values, plaintext fixed) after 1 / 2 / 16 rounds	0–1 / 0–1 / 0–1 of 64 bytes balanced	[Experiment]
Tweak integral, v1.1 (two tweak bytes, 65 536 tweaks), 1 round	0 of 64	[Experiment]
Joint difference in plaintext and tweak chosen to cancel the difference of RK_0, v1.1, rounds 1 / 2 / 3	0.23 / 0.28 / 0.26 bytes with zero difference	[Experiment]
Same with **no** mixing after the tweak (weakened variant), rounds 1 / 2 / 3	56.5 / 0.59 / 0.27	[Experiment]
16 rounds, tweak = 0, 1, 2, …, plaintext 0, 10 000 blocks (v1.1)	monobit z −0.74, runs z 0.78, byte χ^2 261, nibble χ^2 6.1, all below the 99% thresholds	[Experiment]

Statistical tests detect only gross defects and are not evidence of security.

7. The key-schedule weakness of v1.0 and the change in v1.1

7.1 The weakness

In v1.0 every schedule step is a bijection, and the steps after the tweak have no feed-forward. **Given three consecutive round keys RK_i, RK_(i+1), RK_(i+2) of one block, and its tweak, the master key follows immediately. [Experiment]**

1. RK_i gives bytes 0..63 of G_i. These determine the outputs of θ on rows 0..7 of step i+1, so 13 of the 16 bytes 64..79 of G_(i+1) are known. Bytes 70, 71 and 79 remain unknown.

2. RK_(i+2) gives 6 byte equations on row 8 and 7 on row 9 of G_(i+1). An exhaustive search over 2^16 and 2^8 values gives a unique G_(i+1).
3. The steps F_i are inverted back to G_0. The 4 tweak-mixing steps are undone, the tweak is removed, and the 4 key-only steps are undone. The 230 known padding bits confirm the key.

In 3 out of 3 trials the master key was recovered in about 0.1 s, with a single candidate. With **two** consecutive round keys, 3 bytes remain unknown: 2^24 candidates, each verifiable through the padding bits **[Estimate]**. Once the key is known, the round keys of **every** other tweak follow. The per-block "game" therefore did not isolate blocks. This weakness does not break 16 rounds, since no method is known to obtain round keys there. It does mean that, in v1.0, any attack recovering two or three consecutive round keys of one block becomes a full break.

7.2 The change

Version 1.1 adds a Davies–Meyer feed-forward after the tweak injection and doubles the number of steps there from 4 to 8: $G_0 = F(X) \oplus X$, with F made of 8 steps and $X = K^* \oplus$ tweak. The state G_0 of an attacked block can still be reconstructed. But going from G_0 back to X now requires inverting the map $X \to F(X) \oplus X$, a preimage problem on 640 bits. The step-by-step inversion of Section 7.1 no longer applies, because the feed-forward mixes the input into the output. **Whether the map can be inverted by other means is not proven either way**: the feed-forward is a design measure, and its one-wayness is Conjecture 2. The round function is unchanged.

7.3 Evaluation

Attack of 7.1 (3 trials each)	G_0 of the attacked block reconstructed	Master key recovered	Round keys of another tweak predicted
v1.0	3/3	**3/3**	**3/3**
v1.1	3/3	**0/3**	**0/3**

Cost: v1.1 is 13% slower than v1.0 in our implementation (2 295 against 2 651 blocks per second, with a new tweak for every block and the key preparation K* shared).

This is not a proof. The three failures show only that the direct inversion of Section 7.1 no longer works. They do not show that $F(X) \oplus X$ cannot be inverted by other methods, such as meet-in-the-middle or rebound techniques of the kind applied to AES-based hashing modes [Sas11]. Our hypothesis is stated as Conjecture 2.

8. Reduced-round attacks (fixed tweak)

Lemma 4 (3-round integral distinguisher). [Proof + Experiment] Under a fixed tweak, take 256 plaintexts in which one byte takes all values and the rest is constant. After 3 rounds, every one of the 64 output bytes sums to zero.

Proof. After round 1, the row that receives the active byte contains 8 bytes of the form $c \oplus m \cdot S(v)$ with $m \neq 0$, so each takes all 256 values. In round 2, π_2 sends these 8 bytes to 8 distinct rows (Lemma 1). Each row then has one byte that takes all values and 7 constant bytes, so after θ all 64 bytes take all values. In round 3, θ outputs XOR combinations of such bytes, so every output byte sums to zero. Adding the key keeps this property. ∎

Measured: 10 out of 10 trials balanced after 2 and 3 rounds. After 4 rounds 0–1 of 64 bytes were balanced, and 0–1 also with two aligned bytes (2^16 texts). With a fresh tweak per plaintext the property disappears **[Experiment]**.

4-round key recovery. [Experiment] Lemma 4, plus guessing each byte of $\theta^{-1}(RK_4)$ separately (since $\theta^{-1}(C) = \pi_4(S(X_3)) \oplus \theta^{-1}(RK_4)$ and every byte of X_3 is balanced), recovered RK_4 (512 bits) in 5 out of 5 trials with 512–768 chosen plaintexts and in under one second. **This attack recovers only**

RK_4, not the master key. Under v1.0, the master key would follow from three consecutive round keys of one block, or from two with about 2^24 work (Section 7.1). Removing round 4 with the recovered RK_4 and attacking the remaining rounds in the same way might also give RK_3 and RK_2, but we did not implement this. Whether a 4-round master-key recovery on v1.0 is practical therefore remains **[Open]**. Under v1.1, round keys of one tweak do not lead to the master key by the direct method of Section 7.1.

5 rounds. [Estimate] Adding one more round requires guessing 8 bytes of θ^-1(RK_5) and one byte of θ^-1(RK_4) per target byte: about 2^72 guesses with 2^8 texts, plus the unknown row structure of π_4 and π_5. Done naively, this is about 2^80 S-box evaluations. Partial-sum techniques may reduce it. We did not implement this attack, and the estimate may be wrong in either direction.

Margin. The best attack we executed covers 4 of 16 rounds. Our rule of thumb $R \geq 2 \cdot \max(R1, R2, R3)$, a heuristic and not a proof, is satisfied. Here R1 = 2 rounds to full diffusion, R2 = 6 rounds until the best trail is below 2^-512 (by Theorem 1), and R3 = 4–5 rounds attacked, so $2 \cdot 6 = 12 \leq 16$.

9. Security claims and conjectures

What is proven: Lemmas 1–4, Theorems 1 and 2 and their corollaries, as stated, including their limits (Section 5).

Conjecture 1 (security target). In the single-key setting, with chosen plaintexts, ciphertexts and tweaks (repeated tweaks included), 16-round BRI-410 v1.1 has no distinguishing or key-recovery attack with time below 2^256 and data below 2^128 blocks per key. This is a research hypothesis and an invitation to analysis, **not a demonstrated result**. We deliberately set this target far below the key length, and the key length remains 410 bits. The 410-bit key is a symbolic choice, and our analysis covers only the attack classes of Sections 5–8.

Conjecture 2 (tweak isolation). The map $X \to F(X) \oplus X$ with its 8 steps is one-way in practice. As a result, knowing all round keys and permutations for one tweak gives no practical advantage in computing those of another tweak or the master key. This is a hypothesis supported only by the design rationale and by the failure of the direct attack (Section 7.3).

BRI-410 AE. If Conjecture 1 holds, the security theorem of ΘCB3 [KR11] gives confidentiality and authenticity against nonce-respecting adversaries. The forgery probability per attempt is about 2^-416 plus the advantage against the cipher. Limits: messages of at most $2^{28} - 1$ blocks. With random 96-bit nonces, at most 2^32 messages per key, which keeps the probability of a repeated nonce around 2^-33.

Not claimed:

- security against related keys
- security of BRI-410 inside a hash function (known or chosen key)
- resistance to nonce reuse in BRI-410 AE (a repeated nonce reveals equal blocks)
- protection of the reference code against timing and other side channels (it is not constant-time)

10. Design history and negative results

Version	Description	What we found	Action
BRI-410/4, first design	4-bit cells, 256-bit block, "game" (π) derived from the key only, α present	α commutes with the round (Lemma 2): 1 792 equivalent pairs	α removed
BRI-410/4, game per key	as above	the game hides only 1 bit per round at row level (destination row depends only on dr)	documented
BRI-410/4, game per key	as above	integral distinguisher over **4 rounds with 2^12 texts**, **5-round key recovery in under 1 s**	game per block (tweak) introduced: the same attack failed 0/3
BRI-410/4 with tweak	32 rounds	the 4-bit S-box (degree 3) lets the integral property survive one round more than in AES-like designs	replaced by 8-bit cells
BRI-410/8 v1.0	8-bit cells, 512-bit block, 16 rounds	invertible schedule: 3 round keys → master key in 0.1 s (Section 7)	v1.1 (feed-forward, 8 steps)
BRI-410/8 v1.1	this paper	—	open for analysis

Errors corrected from version 1 of this paper.

- It stated a security claim at the level of 2^410 that our analysis did not support.
- It called the second implementation "independent", although it was written separately by the same author.
- It called the CP-SAT values "tight" without saying that they are tight only within the model.
- It reported "49.9%" for a value whose measurement is 49.8%.
- It described cipher version v1.0.

11. Reproducibility and performance

The repository contains:

- the research implementation (Python, standard library)
- a separate single-file reference implementation in English, which agrees bit for bit with the research implementation
- test vectors for v1.0 and v1.1
- 46 unit tests
- every experiment script, with fixed seeds

The CP-SAT model requires OR-Tools. In pure Python the implementation runs at about 2 300 blocks per second with a new tweak for every block and the key preparation K* shared, and BRI-410 AE at about 150 KB/s. A constant-time implementation in C is an open task.

12. Open problems and targets for analysis

1. **Differential and linear:** clustering of differentials, linear hulls, boomerang and impossible-differential attacks; re-running the CP-SAT bounds independently.
2. **Algebraic and integral:** division-property analysis for 4 or more rounds, and algebraic attacks on reduced rounds.
3. **Key schedule:** preimages of $X \to F(X) \oplus X$ with 8 steps (Conjecture 2); related-key attacks; whether reduced-round attacks lead to the master key.
4. **Tweak:** related-tweak attacks that combine tweak and plaintext differences, including pairs whose permutations differ.
5. **Mode and implementation:** an independent check of the ΘCB3 instantiation, and constant-time implementations.

Targets. Distinguishers for 5, 6, 7 or 8 rounds; recovery of round keys for 5 to 8 rounds; recovery of the master key, or of the round keys of a second tweak, from those of one tweak, in either the fixed-tweak or the chosen-tweak setting. The author will acknowledge every result publicly.

Intellectual property

BRI-410 is not patented, and the author does not intend to apply for any patent on it. The specification, the implementations, the test vectors and the analysis scripts are dedicated to the public domain under CC0 1.0.

Acknowledgements

The design ideas and decisions are the author's: the chess analogy and the 410-bit key, the per-block "game played only once", the change to 8-bit cells and 16 rounds, the 416-bit tag and the change of the schedule in v1.1. The author used an AI model (Claude, by Anthropic) as an assistant to write the implementations, run the experiments and computations, and draft this paper. Every result can be reproduced with the published code, and the author is responsible for the content.

References

- [AB00] M. Abdalla, M. Bellare. Increasing the lifetime of a key: a comparative analysis of the security of re-keying techniques. *ASIACRYPT 2000*.
- [All94] L. V. Allis. *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis, University of Limburg, Maastricht, 1994.
- [BJK+16] C. Beierle, J. Jean, S. Kölbl, G. Leander, A. Moradi, T. Peyrin, Y. Sasaki, P. Sasdrich, S. M. Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. *CRYPTO 2016*.
- [BS91] E. Biham, A. Shamir. Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, 1991.
- [DKR97] J. Daemen, L. Knudsen, V. Rijmen. The block cipher Square. *FSE 1997*.
- [DR02] J. Daemen, V. Rijmen. *The Design of Rijndael*. Springer, 2002.
- [FLS+10] N. Ferguson, S. Lucks, B. Schneier, D. Whiting, M. Bellare, T. Kohno, J. Callas, J. Walker. The Skein hash function family. SHA-3 submission, version 1.3, 2010.
- [JNP14] J. Jean, I. Nikolić, T. Peyrin. Tweaks and keys for block ciphers: the TWEAKEY framework. *ASIACRYPT 2014*.
- [JNPS21] J. Jean, I. Nikolić, T. Peyrin, Y. Seurin. The Deoxys AEAD family. *Journal of Cryptology*, 2021.
- [KR11] T. Krovetz, P. Rogaway. The software performance of authenticated-encryption modes. *FSE 2011*.
- [LRW02] M. Liskov, R. Rivest, D. Wagner. Tweakable block ciphers. *CRYPTO 2002*.
- [Mat93] M. Matsui. Linear cryptanalysis method for DES cipher. *EUROCRYPT 1993*.
- [MS77] F. J. MacWilliams, N. J. A. Sloane. *The Theory of Error-Correcting Codes*. North-Holland, 1977.
- [MWGP11] N. Mouha, Q. Wang, D. Gu, B. Preneel. Differential and linear cryptanalysis using mixed-integer linear programming. *Inscrypt 2011*.
- [OGK+15] R. Oliynykov, I. Gorbenko, O. Kazymyrov et al. A new encryption standard of Ukraine: the Kalyna block cipher. *Cryptology ePrint Archive*, 2015/650.
- [PGV93] B. Preneel, R. Govaerts, J. Vandewalle. Hash functions based on block ciphers: a synthetic approach. *CRYPTO 1993*.
- [Sas11] Y. Sasaki. Meet-in-the-middle preimage attacks on AES hashing modes and an application to Whirlpool. *FSE 2011*.
- [Sch93] B. Schneier. Description of a new variable-length key, 64-bit block cipher (Blowfish). *FSE 1993*.
- [SKW+98] B. Schneier, J. Kelsey, D. Whiting, D. Wagner, C. Hall, N. Ferguson. Twofish: a 128-bit block cipher. AES submission, 1998.
- [Sha50] C. E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 1950.
- [Tod15] Y. Todo. Structural evaluation by generalized integral property. *EUROCRYPT 2015*.

Appendix A. Constants

S-box S (row = high nibble, column = low nibble, hexadecimal):

```
15 86 4d a2 24 91 fa cd 75 c1 7e 8d cb c8 f6 34
6d f5 22 ff a0 6a 80 12 ab 87 0b 9d 1f 56 5f df
2a 84 bf 1b 28 d5 36 30 1e 2f 33 6b 10 68 0c 99
d3 fb 09 90 0f ad aa 42 9c bd c3 17 ea 62 38 7b
45 8c a9 39 40 83 25 3d a1 be b1 41 bb 6e e6 a8
d8 95 b8 b7 46 fd 77 5c e8 14 d7 88 16 4b 13 7f
ec 81 8f d9 eb 32 3a d1 b6 ef 8e c9 97 9a 50 b9
ee 72 a4 63 c6 66 f1 00 e1 53 5e 3f 8b b2 5d da
49 57 c4 ce 37 d4 cf af b4 19 b0 e3 31 01 76 26
5a ba 04 03 70 f4 f0 47 f9 f8 8a e0 35 4e 73 27
21 43 c7 0a 59 e5 2d 78 e9 a3 d2 6c 89 e2 4a 94
05 5b 51 cc 55 fe 7d c0 b5 ca 79 93 48 db e4 3c
bc ed 54 9f 64 60 65 82 a5 7a 02 2b 6f 98 08 71
69 ac 1c 07 20 f7 4f 61 23 67 b3 f3 9e a6 1d 29
58 96 74 f2 a7 11 1a 9b 3b 52 e7 dc 06 85 c2 4c
2c 44 3e 2e ae 7c 92 de dd 0d d0 18 0e fc c5 d6
```

Matrix M (θ) and **M^-1** (θ^-1), hexadecimal, row by row:

```
M                       M^-1
6a d7 84 ac af d6 e3 61    f1 a7 cf 26 40 46 f8 48
de e3 72 e6 d3 eb d7 17    36 69 5f 5b d4 15 11 c7
6d 08 d8 1b 16 b1 4e 51    4d b9 41 23 50 a3 55 92
32 4e 8a fc 5e 0d 08 ec    80 53 52 4d f4 8b 31 19
8a 1e 32 53 36 cf b3 22    ed 73 15 f2 78 4f aa 26
d8 b3 6d 04 43 ad 1e f0    99 ae e0 09 6f d6 1e 68
72 5d de a9 49 c6 50 18    e6 e2 05 06 d1 2e 74 a8
84 50 6a 27 a6 0e 5d 3e    2b 85 a1 97 df 11 f5 3e
```

Padding constant (bits 410..639 of L, as an integer):
`0x391d69042f080a4aa81f1dc9676116fafcd238b21e43a01e5fb06f0fe6`

Appendix B. Test vectors (v1.1, hexadecimal)

```
Key        = 00 × 52     Tweak = 00 × 16     Plaintext = 00 × 64
Ciphertext = 67b7e691273da3b40bedf3cf4abad7a3c960473b1ef4356d961413423c718053
             2fab02454f11cb600955edd094de6490024eb8f06f40d4f0d30443e004069d72

Key        = 000102…2f303132 02 (bytes 0..50 counting, byte 51 = 02)
Tweak      = 000102030405060708090a0b0c0d0e0f     Plaintext = 000102…3e3f
Ciphertext = 9f52662ad01ba4bafe6fa894e81fd6b23fa22078bb5054587d09214cd8306ba1
             c3bf6167b07494d2d5f379c26b991a79403de492f4e310639917d33af96ada4c
```

12 block-cipher vectors (with RK_0 and RK_16) and 6 BRI-410 AE vectors are in `reference/test_vectors.json`.

Appendix C. The CP-SAT model

The variables are x[t][c] ∈ {0,1}: cell c is active at the input of the S-layer of round t. For every round t and row q, one binary variable d[t][q] marks the row as active. Every active input or output cell of the row implies d[t][q], and the constraint is (active inputs of θ_t in row q) + (active outputs x[t+1][8q..8q+7]) ≥ 9·d[t][q]. The inputs of row q are the cells c with r + k·dr ≡ q, where c = (r, k). A binary variable s[t] selects dr = +1 or −1 in round t, so the solver chooses the worst case. The objective minimises Σ x[t][c] over rounds 1..R, subject to Σ x[0][c] ≥ 1. The model of the schedule is the same, with 10 rows and the map j → 9j mod 80, and it starts from a non-zero difference in bytes 0..15 (the tweak). Code: `matje/f4b_milp.py`.